

SV Project_ Synchronous FIFO

Supervised by : ENG/ Kareem waseem
Prepared by : Amr alhassan yacoob

1. RTL CODE & Assertions:

```
////////////////////////////////////
////////
// Author      : Kareem Waseem
// Course      : Digital Verification using SV & UVM
// Description: FIFO RTL Design with Assertions & Cover Properties
////////////////////////////////////
////////

module FIFO(FIFO_if.DUT fifoif);

// =====
// PARAMETERS & LOCAL CONSTANTS
// =====
parameter FIFO_WIDTH = 16;          // Data bus width
parameter FIFO_DEPTH = 8;           // FIFO depth
localparam max_fifo_addr = $clog2(FIFO_DEPTH); // Address width for
pointers

// =====
// INTERNAL SIGNALS
// =====
logic [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0]; // FIFO storage memory
logic [max_fifo_addr-1:0] wr_ptr, rd_ptr;    // Write & read pointers
logic [max_fifo_addr:0] count;               // FIFO element counter

// =====
// WRITE LOGIC
// =====
always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
    if (!fifoif.rst_n) begin
        wr_ptr <= 0;
        fifoif.wr_ack <= 0;
        fifoif.overflow <= 0;
    end
    else if (fifoif.wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= fifoif.data_in; // Write data to memory
        fifoif.wr_ack <= 1;           // Acknowledge write
        wr_ptr <= wr_ptr + 1;         // Increment write pointer
    end
    else begin
        fifoif.wr_ack <= 0;
        if (fifoif.full && fifoif.wr_en)
            fifoif.overflow <= 1;    // Overflow if write attempted
on full FIFO
        else
            fifoif.overflow <= 0;
    end
end
```

```

        end
    end

    // =====
    // READ LOGIC
    // =====
    always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
        if (!fifoif.rst_n) begin
            rd_ptr <= 0;
            fifoif.wr_ack <= 0;
            fifoif.underflow <= 0;
        end
        else if (fifoif.rd_en && count != 0) begin
            fifoif.data_out <= mem[rd_ptr]; // Read data from memory
            rd_ptr <= rd_ptr + 1;           // Increment read pointer
        end
        else begin
            if (fifoif.empty && fifoif.rd_en)
                fifoif.underflow <= 1; // Underflow if read attempted
on empty FIFO
            else
                fifoif.underflow <= 0;
        end
    end

    // =====
    // COUNTER UPDATE LOGIC
    // =====
    always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
        if (!fifoif.rst_n) begin
            count <= 0;
        end
        else begin
            // Write only
            if ({fifoif.wr_en, fifoif.rd_en} == 2'b10 && !fifoif.full)
                count <= count + 1;
            // Read only
            else if ({fifoif.wr_en, fifoif.rd_en} == 2'b01 &&
!fifoif.empty)
                count <= count - 1;
            // Simultaneous write & read when FIFO empty (increase count)
            else if ({fifoif.wr_en, fifoif.rd_en} == 2'b11 && fifoif.empty)
                count <= count + 1;
            // Simultaneous write & read when FIFO full (decrease count)
            else if ({fifoif.wr_en, fifoif.rd_en} == 2'b11 && fifoif.full)
                count <= count - 1;
        end
    end
end

```

```

// =====
// STATUS FLAGS
// =====
assign fifoif.full      = (count == FIFO_DEPTH);
assign fifoif.empty     = (count == 0);
assign fifoif.almostfull = (count == FIFO_DEPTH-1);
assign fifoif.almostempty = (count == 1);

// =====
// ASSERTION PROPERTIES
// =====

// 1. Reset behavior
property Reset_Behavior;
    @(posedge fifoif.clk) !fifoif.rst_n |-> (!wr_ptr && !rd_ptr &&
!count);
endproperty

// 2. Write acknowledge
property Write_Acknowledge;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (fifoif.wr_en && !fifoif.full) |=> fifoif.wr_ack;
endproperty

// 3. Overflow detection
property Overflow_Detection;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (fifoif.full && fifoif.wr_en) |=> fifoif.overflow;
endproperty

// 4. Underflow detection
property Underflow_Detection;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (fifoif.empty && fifoif.rd_en) |=> fifoif.underflow;
endproperty

// 5. Empty flag correctness
property Empty_Flag;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (count == 0) |-> fifoif.empty;
endproperty

// 6. Full flag correctness
property Full_Flag;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (count == FIFO_DEPTH) |-> fifoif.full;
endproperty

```

```

// 7. Almost full correctness
property Almost_Full;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (count == FIFO_DEPTH-1) |-> fifoif.almostfull;
endproperty

// 8. Almost empty correctness
property Almost_empty;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        (count == 1) |-> fifoif.almostempty;
endproperty

// 9. Pointer wraparound for write pointer
property Pointer_Wraparound_write;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        ((wr_ptr == FIFO_DEPTH-1) && fifoif.wr_en && !fifoif.full) | =>
!wr_ptr[->1];
endproperty

// 10. Pointer wraparound for read pointer
property Pointer_Wraparound_read;
    @(posedge fifoif.clk) disable iff (!fifoif.rst_n)
        ((rd_ptr == FIFO_DEPTH-1) && fifoif.rd_en && !fifoif.empty) | =>
!rd_ptr[->1];
endproperty

// 11. Pointer & count threshold
property Pointer_threshold;
    @(posedge fifoif.clk) disable iff(!fifoif.rst_n)
        ##5 (count <= FIFO_DEPTH) && (wr_ptr <= FIFO_DEPTH) && (rd_ptr
<= FIFO_DEPTH);
endproperty

// =====
// ASSERT & COVER INSTANTIATION
// =====
assert property(Reset_Behavior);
assert property(Write_Acknowledge);
assert property(Overflow_Detection);
assert property(Underflow_Detection);
assert property(Empty_Flag);
assert property(Full_Flag);
assert property(Almost_Full);
assert property(Almost_empty);
assert property(Pointer_Wraparound_write);
assert property(Pointer_Wraparound_read);
assert property(Pointer_threshold);

```

```

cover property(Reset_Behavior);
cover property(Write_Acknowledge);
cover property(Overflow_Detection);
cover property(Underflow_Detection);
cover property(Empty_Flag);
cover property(Full_Flag);
cover property(Almost_Full);
cover property(Almost_empty);
cover property(Pointer_Wraparound_write);
cover property(Pointer_Wraparound_read);
cover property(Pointer_threshold);

// =====
// EXTRA ASSERTIONS (active only in simulation)
// =====
`ifdef SIM

// Full flag check
always @(posedge fifoif.clk) begin
    if (!fifoif.rst_n) disable full_flag_check;
    else full_flag_check: assert (fifoif.full == (count == FIFO_DEPTH))
        else $error("full flag mismatch: full=%0b, count=%0d",
fifoif.full, count);
end

// Empty flag check
always @(posedge fifoif.clk) begin
    if (!fifoif.rst_n) disable empty_flag_check;
    else empty_flag_check: assert (fifoif.empty == (count == 0))
        else $error("empty flag mismatch: empty=%0b, count=%0d",
fifoif.empty, count);
end

// Almost full check
always @(posedge fifoif.clk) begin
    if (!fifoif.rst_n) disable almostfull_flag_check;
    else almostfull_flag_check: assert (fifoif.almostfull == (count ==
FIFO_DEPTH-1))
        else $error("almostfull flag mismatch: almostfull=%0b,
count=%0d", fifoif.almostfull, count);
end

// Almost empty check
always @(posedge fifoif.clk) begin
    if (!fifoif.rst_n) disable almostempty_flag_check;
    else almostempty_flag_check: assert (fifoif.almostempty == (count
== 1))

```

```

        else $error("almostempty flag mismatch: almostempty=%0b,
count=%0d", fifoif.almostempty, count);
    end

    // Underflow check
    always @(posedge fifoif.clk) begin
        if (!fifoif.rst_n) disable underflow_flag_check;
        else underflow_flag_check: assert (fifoif.underflow ==
(fifoif.empty && fifoif.rd_en))
            else $error("underflow flag mismatch: underflow=%0b, empty=%0b,
rd_en=%0b", fifoif.underflow, fifoif.empty, fifoif.rd_en);
    end

    // Count should never exceed FIFO depth
    always @(posedge fifoif.clk) begin
        if (!fifoif.rst_n) disable count_range_check;
        else count_range_check: assert (count <= FIFO_DEPTH)
            else $error("count exceeded FIFO_DEPTH: count=%0d", count);
    end

    // Count should never go negative
    always @(posedge fifoif.clk) begin
        if (!fifoif.rst_n) disable count_nonneg_check;
        else count_nonneg_check: assert (count >= 0)
            else $error("count negative: count=%0d", count);
    end

`endif

endmodule

```

2. The Interface:

```

interface FIFO_if(clk);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    input clk;
    logic [FIFO_WIDTH-1:0] data_in;
    logic rst_n, wr_en, rd_en;
    logic wr_ack, overflow, underflow;
    logic [FIFO_WIDTH-1:0] data_out;
    logic full, empty, almostfull, almostempty;
    event etrigger;
    modport DUT (
        input clk, rst_n, wr_en, rd_en, data_in,

```

```

    output full, empty, almostfull, almostempty,wr_ack,overflow,
    underflow, data_out);
modport monitor (
    input clk, rst_n, wr_en, rd_en, data_in, full, empty,
    almostfull,wr_ack,ettrigger,overflow, almostempty, underflow, data_out);
modport TB (
    input clk, full, empty,wr_ack,overflow, almostfull, almostempty,
    underflow, data_out,
    output rst_n, wr_en, rd_en, data_in,ettrigger);
endinterface

```

3. Package for Testing FIFO:

```

package shared_pkg;
int correct_count,error_count;
bit test_finished;
endpackage : shared_pkg

```

4. Package for Transaction:

```

package transaction_pkg;
import shared_pkg::*;
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
class FIFO_transaction;
rand bit [FIFO_WIDTH-1:0] data_in;
rand bit rst_n, wr_en, rd_en;
bit [FIFO_WIDTH-1:0] data_out;
bit wr_ack, overflow;
bit full, empty, almostfull, almostempty, underflow;
int RD_EN_ON_DIST, WR_EN_ON_DIST;

//constructor
function new(int RD_EN_ON_DIST=30, int WR_EN_ON_DIST=70);
    this.RD_EN_ON_DIST = RD_EN_ON_DIST;
    this.WR_EN_ON_DIST = WR_EN_ON_DIST;
endfunction : new

//constraints
constraint c_rst_n {rst_n dist {0:=5,1:=95}; }
constraint c_wr_en {wr_en dist {0:=100-WR_EN_ON_DIST,1:=WR_EN_ON_DIST};
}
constraint c_rd_en {rd_en dist {0:=100-
RD_EN_ON_DIST,1:=RD_EN_ON_DIST}};

```



```

endclass : FIFO_transaction
endpackage :transaction_pkg

```

5. Package For scoreboard:

```

package scoreboard_pkg;
import transaction_pkg::*;
import shared_pkg::*;
bit [FIFO_WIDTH-1:0] data_out_ref;
bit [FIFO_WIDTH-1:0] mem_ref[FIFO_DEPTH-1:0];
bit [2:0] write_ptr_ref, read_ptr_ref;
int count;
bit full_ref, empty_ref, almostfull_ref, almostempty_ref;
bit wr_ack_ref, overflow_ref, underflow_ref;

class FIFO_scoreboard;
task reference_model(FIFO_transaction F_scr_txn2 );
    // Simple reference model logic for FIFO
    if (F_scr_txn2.rst_n == 0) begin
        full_ref = 0;
        empty_ref = 1;
        wr_ack_ref = 0;
        overflow_ref = 0;
        underflow_ref = 0;
        write_ptr_ref = 0;
        read_ptr_ref = 0;
        count = 0;
    end else begin
        if (F_scr_txn2.wr_en && !full_ref && count < FIFO_DEPTH) begin
            // Simulate write operation
            wr_ack_ref = 1;
            empty_ref = 0;
            mem_ref[write_ptr_ref]= F_scr_txn2.data_in; // Store data
in reference memory
            write_ptr_ref = write_ptr_ref + 1;
            count++;
            if (count==FIFO_DEPTH-1) begin
                almostfull_ref = 1;
            end
            if (count==FIFO_DEPTH) begin
                full_ref = 1;
            end
        end else begin
            wr_ack_ref = 0;
            if (F_scr_txn2.wr_en && full_ref) begin

```

```

        overflow_ref = 1; // Overflow condition
    end else begin
        overflow_ref = 0;
    end
end

if (F_scr_txn2.rd_en && count !=0) begin
    // Simulate read operation
    data_out_ref =mem_ref[read_ptr_ref]; // Read data from
reference memory;
    read_ptr_ref = read_ptr_ref + 1;
    count--;
    if (count==1) begin
        almostempty_ref = 1;
    end
    if (count==0) begin
        empty_ref = 1;
    end
end else begin
    if (F_scr_txn2.rd_en && empty_ref) begin
        underflow_ref = 1; // Underflow condition
    end else begin
        underflow_ref = 0;
    end
end
if(F_scr_txn2.rd_en && F_scr_txn2.wr_en && !full_ref)begin
    wr_ack_ref = 1;
    empty_ref = 0;
    mem_ref[write_ptr_ref]= F_scr_txn2.data_in; // Store data
in reference memory
    write_ptr_ref = write_ptr_ref + 1;
    count++;
end else if(F_scr_txn2.rd_en && F_scr_txn2.wr_en && !empty_ref) begin
    data_out_ref =mem_ref[read_ptr_ref]; // Read data from
reference memory;
    read_ptr_ref = read_ptr_ref + 1;
    count--;
end else if(F_scr_txn2.rd_en && F_scr_txn2.wr_en && empty_ref) begin
    wr_ack_ref = 1;
    empty_ref = 0;
    mem_ref[write_ptr_ref]= F_scr_txn2.data_in; // Store data
in reference memory
    write_ptr_ref = write_ptr_ref + 1;
    count++;
end else if(F_scr_txn2.rd_en && F_scr_txn2.wr_en && full_ref)begin
    data_out_ref =mem_ref[read_ptr_ref]; // Read data from
reference memory;
    read_ptr_ref = read_ptr_ref + 1;

```

```

        count--;
    end
    end
endtask
task check_data(FIFO_transaction F_scr_txn);
    reference_model(F_scr_txn);
    if ((F_scr_txn.data_out != data_out_ref) &&
        (F_scr_txn.full != full_ref) &&
        (F_scr_txn.empty != empty_ref) &&
        (F_scr_txn.almostfull != almostfull_ref) &&
        (F_scr_txn.almostempty != almostempty_ref) &&
        (F_scr_txn.wr_ack != wr_ack_ref) &&
        (F_scr_txn.overflow != overflow_ref) &&
        (F_scr_txn.underflow != underflow_ref)) begin
        error_count++;
        $display("SCOREBOARD ERROR at time %0t: Expected:
data_out=%0h, full=%0b, empty=%0b, almostfull=%0b, almostempty=%0b,
wr_ack=%0b, overflow=%0b, underflow=%0b | Got: data_out=%0h, full=%0b,
empty=%0b, almostfull=%0b, almostempty=%0b, wr_ack=%0b, overflow=%0b,
underflow=%0b",
            $time, data_out_ref, full_ref, empty_ref,
almostfull_ref, almostempty_ref, wr_ack_ref, overflow_ref,
underflow_ref,
            F_scr_txn.data_out, F_scr_txn.full, F_scr_txn.empty,
F_scr_txn.almostfull, F_scr_txn.almostempty, F_scr_txn.wr_ack,
F_scr_txn.overflow, F_scr_txn.underflow);
        end else begin
            correct_count++;
        end
    endtask
endclass : FIFO_scoreboard

endpackage : scoreboard_pkg

```

6. Package For coverage:

```

package coverage_pkg;
    import transaction_pkg::*;
    class FIFO_coverage;
        FIFO_transaction F_cvg_txn;
    //covergroups
        covergroup FIFO_cvg_grp();
            write_enable_cp : coverpoint F_cvg_txn.wr_en{
                bins one_wr_en={1};
                bins zero_wr_en={0};
            }
            read_enable_cp : coverpoint F_cvg_txn.rd_en{

```

```

        bins one_rd_en={1};
        bins zero_rd_en={0};
    }
    full_cp : coverpoint F_cvg_txn.full{
        bins one_full={1};
        bins zero_full={0};
    }
    empty_cp: coverpoint F_cvg_txn.empty{
        bins one_empty={1};
        bins zero_empty={0};
    }
    almost_full_cp : coverpoint F_cvg_txn.almostfull{
        bins one_almostfull={1};
        bins zero_almostfull={0};
    }
    almost_empty_cp: coverpoint F_cvg_txn.almostempty{
        bins one_almostempty={1};
        bins zero_almostempty={0};
    }
    overflow_cp: coverpoint F_cvg_txn.overflow{
        bins one_overflow={1};
        bins zero_overflow={0};
    }
    underflow_cp: coverpoint F_cvg_txn.underflow{
        bins one_underflow={1};
        bins zero_underflow={0};
    }
    wr_ack_cp: coverpoint F_cvg_txn.wr_ack{
        bins one_wr_ack={1};
        bins zero_wr_ack={0};
    }
}
// Crosses

    wr_rd_full_cross      : cross write_enable_cp, read_enable_cp,
full_cp{
    ignore_bins
full_0=binsof(write_enable_cp.zero_wr_en)&&binsof(read_enable_cp.one_rd_
_en)&&binsof(full_cp.one_full);
    ignore_bins
full_1=binsof(write_enable_cp.one_wr_en)&&binsof(read_enable_cp.one_rd_
_en)&&binsof(full_cp.one_full);
}
    wr_rd_empty_cross      : cross write_enable_cp, read_enable_cp,
empty_cp;
    wr_rd_almost_full_cross: cross write_enable_cp, read_enable_cp,
almost_full_cp;
    wr_rd_almost_empty_cross: cross write_enable_cp, read_enable_cp,
almost_empty_cp;

```

```

        wr_rd_overflow_cross : cross write_enable_cp, read_enable_cp,
overflow_cp{
            ignore_bins
ov_0=binsof(write_enable_cp.zero_wr_en)&&binsof(read_enable_cp.one_rd_e
n)&&binsof(overflow_cp.one_overflow);
            ignore_bins
ov_1=binsof(write_enable_cp.zero_wr_en)&&binsof(read_enable_cp.zero_rd_
en)&&binsof(overflow_cp.one_overflow);
        }
        wr_rd_underflow_cross : cross write_enable_cp, read_enable_cp,
underflow_cp{
            ignore_bins
under_0=binsof(write_enable_cp.one_wr_en)&&binsof(read_enable_cp.zero_r
d_en)&&binsof(underflow_cp.one_underflow);
            ignore_bins
under_1=binsof(write_enable_cp.zero_wr_en)&&binsof(read_enable_cp.zero_
rd_en)&&binsof(underflow_cp.one_underflow);
        }
        wr_rd_wrack_cross : cross write_enable_cp, read_enable_cp,
wr_ack_cp{
            ignore_bins
wrack_0=binsof(write_enable_cp.zero_wr_en)&&binsof(read_enable_cp.zero_
rd_en)&&binsof(wr_ack_cp.one_wr_ack);
            ignore_bins
wrack_1=binsof(write_enable_cp.zero_wr_en)&&binsof(read_enable_cp.one_r
d_en)&&binsof(wr_ack_cp.one_wr_ack);
        }
    endgroup : FIFO_cvg_grp

// constructor
    function new();
        F_cvg_txn = new();
        FIFO_cvg_grp= new();
    endfunction

    function void sample_data(FIFO_transaction F_txn);
        F_cvg_txn = F_txn;
        FIFO_cvg_grp.sample();
    endfunction
endclass : FIFO_coverage
endpackage : coverage_pkg

```

7. Package For Monitor:

```

import shared_pkg::*;
import transaction_pkg::*;
import coverage_pkg::*;
import scoreboard_pkg::*;
module FIFO_monitor(FIFO_if.monitor fifoif);
FIFO_transaction trans;
FIFO_coverage cov;
FIFO_scoreboard sb;
initial begin
    trans=new();
    cov=new();
    sb=new();
    forever begin
        wait(fifoif.etrigger.triggered);
        @(negedge fifoif.clk);
        trans.data_in= fifoif.data_in;
        trans.rst_n=fifoif.rst_n;
        trans.wr_en= fifoif.wr_en;
        trans.rd_en=fifoif.rd_en;
        trans.data_out=fifoif.data_out;
        trans.wr_ack= fifoif.wr_ack;
        trans.overflow=fifoif.overflow;
        trans.underflow=fifoif.underflow;
        trans.full=fifoif.full;
        trans.empty=fifoif.empty;
        trans.almostfull=fifoif.almostfull;
        trans.almostempty=fifoif.almostempty;
        fork
            begin
                cov.sample_data(trans);
            end
            begin
                sb.check_data(trans);
            end
        join
    end

    if(test_finished)begin
        $display("errors=%d,correct=%d",error_count,correct_count);
        $stop;
    end
end
endmodule : FIFO_monitor

```

8. TestBench code:

```

import shared_pkg::*;
import transaction_pkg::*;

module FIFO_tb(FIFO_if.TB fifoif);
    FIFO_transaction trans_test;
    initial begin
        error_count=0;
        correct_count=0;
        test_finished=0;
        trans_test=new();
        repeat(10000)begin
            assert(trans_test.randomize());
            fifoif.data_in=trans_test.data_in;
            fifoif.rst_n=trans_test.rst_n;
            fifoif.wr_en=trans_test.wr_en;
            fifoif.rd_en=trans_test.rd_en;
            @(negedge fifoif.clk);
            ->fifoif.etrigger;
        end
        test_finished=1;
    end
endmodule

```

9. Top Module:

```

module FIFO_top();
    bit clk;
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    FIFO_if fifoif(clk);
    FIFO DUT(fifoif);
    FIFO_tb tb(fifoif);
    FIFO_monitor monitor(fifoif);
endmodule : FIFO_top

```

10. Do File:

```
vlib work
vlog -f src_files.list +cover -covercells
vsim -voptargs=+acc work.FIFO_top -cover
add wave /FIFO_top/fifoif/*
coverage save FIFO_top.ucdb -onexit
run -all
```

11. src_files:

FIFO_if.v
FIFO.v
FIFO_pkg.v
FIFO_pkg_transaction.v
FIFO_pkg_coverage.v
FIFO_pkg_scoreboard.v
FIFO_tb.v
top.v

12. snippets For coverage:

```

38 =====
39 === Instance: /FIFO_top/DUT
40 === Design Unit: work.FIFO
41 =====
42
43 Assertion Coverage:
44 | Assertions | 11 | 11 | 0 | 100.00% |
45 |-----|
46 | Name | File(Line) | Failure | Pass |
47 | | | Count | Count |
48 |-----|
49 | /FIFO_top/DUT/assert_pointer_threshold |
50 | | FIFO.sv(180) | 0 | 1 |
51 | /FIFO_top/DUT/assert_pointer_wraparound_read |
52 | | FIFO.sv(179) | 0 | 1 |
53 | /FIFO_top/DUT/assert_pointer_wraparound_write |
54 | | FIFO.sv(178) | 0 | 1 |
55 | /FIFO_top/DUT/assert_almost_empty |
56 | | FIFO.sv(177) | 0 | 1 |
57 | /FIFO_top/DUT/assert_almost_full |
58 | | FIFO.sv(176) | 0 | 1 |
59 | /FIFO_top/DUT/assert_full_flag |
60 | | FIFO.sv(175) | 0 | 1 |
61 | /FIFO_top/DUT/assert_empty_flag |
62 | | FIFO.sv(174) | 0 | 1 |
63 | /FIFO_top/DUT/assert_underflow_detection |
64 | | FIFO.sv(173) | 0 | 1 |
65 | /FIFO_top/DUT/assert_overflow_detection |
66 | | FIFO.sv(172) | 0 | 1 |
67 | /FIFO_top/DUT/assert_write_acknowledge |
68 | | FIFO.sv(171) | 0 | 1 |
69 | /FIFO_top/DUT/assert_reset_behavior |
70 | | FIFO.sv(170) | 0 | 1 |
71 Branch Coverage:
72 | Enabled Coverage | Bins | Hits | Misses | Coverage |
73 |-----|
74 | Branches | 17 | 17 | 0 | 100.00% |
75
76 =====Branch Details=====

```



```

76 =====Branch Details=====
77
78 Branch Coverage for instance /FIFO_top/DUT
79
80 Line      Item      Count      Source
81 ----      -
82 File FIFO.sv
83 -----IF Branch-----
84 27          10466      Count coming in to IF
85 27          949        if (!fifoif.rst_n) begin
86
87 32          4952        else if (fifoif.wr_en && count < FIFO_DEPTH) begin
88
89 37          4565        else begin
90
91 Branch totals: 3 hits of 3 branches = 100.00%
92
93 -----IF Branch-----
94 39          4565      Count coming in to IF
95 39          1634      if (fifoif.full && fifoif.wr_en)
96
97 41          2931      else
98
99 Branch totals: 2 hits of 2 branches = 100.00%
100
101 -----IF Branch-----
102 50          10466      Count coming in to IF
103 50          949        if (!fifoif.rst_n) begin
104
105 55          2540      else if (fifoif.rd_en && count != 0) begin
106
107 59          6977      else begin
108
109 Branch totals: 3 hits of 3 branches = 100.00%
110
111 -----IF Branch-----
112 60          6977      Count coming in to IF
113 60          256        if (fifoif.empty && fifoif.rd_en)
114
115 62          6721      else
116
117 Branch totals: 2 hits of 2 branches = 100.00%
118
119 -----IF Branch-----
120 71          9221      Count coming in to IF
121 71          943        if (!fifoif.rst_n) begin
122
123 74          8278      else begin
124
125 Branch totals: 2 hits of 2 branches = 100.00%
126
127 -----IF Branch-----
128 76          8278      Count coming in to IF
129 76          3497      if ((fifoif.wr_en, fifoif.rd_en) == 2'b10 && !fifoif.full)
130
131 79          779        else if ((fifoif.wr_en, fifoif.rd_en) == 2'b01 && !fifoif.empty)
132
133 82          174        else if ((fifoif.wr_en, fifoif.rd_en) == 2'b11 && fifoif.empty)
134
135 85          480        else if ((fifoif.wr_en, fifoif.rd_en) == 2'b11 && fifoif.full)
136
137 3348      All False Count
138 Branch totals: 5 hits of 5 branches = 100.00%
139
140
141 Condition Coverage:
142 Enabled Coverage      Bins      Covered      Misses      Coverage
143 -----
144 Conditions            20        20          0      100.00%
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

Statements - by instance (/FIFO_top/DUT) Statement

```

26 always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
27   wr_ptr <= 0;
28   fifoif.wr_ack <= 0;
29   fifoif.overflow <= 0;
30   mem[wr_ptr] <= fifoif.data_in; // Write data to memory
31   fifoif.wr_ack <= 1;           // Acknowledge write
32   wr_ptr <= wr_ptr + 1;         // Increment write pointer
33   fifoif.wr_ack <= 0;
34   fifoif.overflow <= 1;         // Overflow if write attempted on full FIFO
35   fifoif.overflow <= 0;
36   always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
37     rd_ptr <= 0;
38     fifoif.wr_ack <= 0;
39     fifoif.underflow <= 0;
40     fifoif.data_out <= mem[rd_ptr]; // Read data from memory
41     rd_ptr <= rd_ptr + 1;           // Increment read pointer
42     fifoif.underflow <= 1;         // Underflow if read attempted on empty FIFO
43     fifoif.underflow <= 0;
44     always @(posedge fifoif.clk or negedge fifoif.rst_n) begin
45       count <= 0;
46       count <= count + 1;
47       count <= count - 1;
48       count <= count + 1;
49       count <= count - 1;
50       assign fifoif.full      = (count == FIFO_DEPTH);
51       assign fifoif.empty     = (count == 0);
52       assign fifoif.almostfull = (count == FIFO_DEPTH-1);
53       assign fifoif.almostempty = (count == 1);

```

Branches - by instance (/FIFO_top/DUT)

```
FIFO.sv
27 if (!fifoif.rst_n) begin
32 else if (fifoif.wr_en && count < FIFO_DEPTH) begin
37 else begin
39 if (fifoif.full && fifoif.wr_en)
41 else
50 if (!fifoif.rst_n) begin
55 else if (fifoif.rd_en && count != 0) begin
59 else begin
60 if (fifoif.empty && fifoif.rd_en)
62 else
71 if (!fifoif.rst_n) begin
74 else begin
76 if ((fifoif.wr_en, fifoif.rd_en) == 2'b10 && !fifoif.full)
79 else if ((fifoif.wr_en, fifoif.rd_en) == 2'b01 && !fifoif.empty)
82 else if ((fifoif.wr_en, fifoif.rd_en) == 2'b11 && fifoif.empty)
85 else if ((fifoif.wr_en, fifoif.rd_en) == 2'b11 && fifoif.full)
```

Toggle - by instance (/FIFO_top/DUT) Toggle

```
sim:/FIFO_top/DUT
  ✓ count
  ✓ rd_ptr
  ✓ wr_ptr
```

1533	COVERGROUP COVERAGE:				
1534					
1535	Covergroup	Metric	Goal	Bins	Status
1536					
1537					
1538	TYPE /coverage_pkg/FIFO_coverage/FIFO_cvg_grp	100.00%	100	-	Covered
1539	covered/total bins:	66	66	-	
1540	missing/total bins:	0	66	-	
1541	% Hit:	100.00%	100	-	
1542	Coverpoint write_enable_cp	100.00%	100	-	Covered
1543	covered/total bins:	2	2	-	
1544	missing/total bins:	0	2	-	
1545	% Hit:	100.00%	100	-	
1546	bin one_wr_en	6936	1	-	Covered
1547	bin zero_wr_en	3064	1	-	Covered
1548	Coverpoint read_enable_cp	100.00%	100	-	Covered
1549	covered/total bins:	2	2	-	
1550	missing/total bins:	0	2	-	
1551	% Hit:	100.00%	100	-	
1552	bin one_rd_en	2929	1	-	Covered
1553	bin zero_rd_en	7071	1	-	Covered
1554	Coverpoint full_cp	100.00%	100	-	Covered
1555	covered/total bins:	2	2	-	
1556	missing/total bins:	0	2	-	
1557	% Hit:	100.00%	100	-	
1558	bin one_full	2489	1	-	Covered
1559	bin zero_full	7511	1	-	Covered
1560	Coverpoint empty_cp	100.00%	100	-	Covered
1561	covered/total bins:	2	2	-	
1562	missing/total bins:	0	2	-	
1563	% Hit:	100.00%	100	-	
1564	bin one_empty	850	1	-	Covered
1565	bin zero_empty	9150	1	-	Covered
1566	Coverpoint almost_full_cp	100.00%	100	-	Covered
1567	covered/total bins:	2	2	-	
1568	missing/total bins:	0	2	-	
1569	% Hit:	100.00%	100	-	
1570	bin one_almostfull	1687	1	-	Covered
1571	bin zero_almostfull	8313	1	-	Covered
1572	Coverpoint almost_empty_cp	100.00%	100	-	Covered
1573	covered/total bins:	2	2	-	
1574	missing/total bins:	0	2	-	
1575	% Hit:	100.00%	100	-	
1576	bin one_almostempty	1036	1	-	Covered
1577	bin zero_almostempty	8964	1	-	Covered
1578	Coverpoint overflow_cp	100.00%	100	-	Covered
1579	covered/total bins:	2	2	-	
1580	missing/total bins:	0	2	-	
1581	% Hit:	100.00%	100	-	
1582	bin one_overflow	2042	1	-	Covered
1583	bin zero_overflow	7958	1	-	Covered
1584	Coverpoint underflow_cp	100.00%	100	-	Covered
1585	covered/total bins:	2	2	-	
1586	missing/total bins:	0	2	-	
1587	% Hit:	100.00%	100	-	
1588	bin one_underflow	315	1	-	Covered
1589	bin zero_underflow	9685	1	-	Covered
1590	Coverpoint wr_ack_cp	100.00%	100	-	Covered
1591	covered/total bins:	2	2	-	
1592	missing/total bins:	0	2	-	
1593	% Hit:	100.00%	100	-	
1594	bin one_wr_ack	4952	1	-	Covered
1595	bin zero_wr_ack	5048	1	-	Covered

1596	Cross wr_rd_full_cross	100.00%	100	-	Covered
1597	covered/total bins:	6	6	-	
1598	missing/total bins:	0	6	-	
1599	% Hit:	100.00%	100	-	
1600	Auto, Default and User Defined Bins:				
1601	bin <zero_wr_en,zero_rd_en,zero_full>	1633	1	-	Covered
1602	bin <zero_wr_en,one_rd_en,zero_full>	896	1	-	Covered
1603	bin <one_wr_en,zero_rd_en,zero_full>	2949	1	-	Covered
1604	bin <one_wr_en,one_rd_en,zero_full>	2033	1	-	Covered
1605	bin <zero_wr_en,zero_rd_en,one_full>	535	1	-	Covered
1606	bin <one_wr_en,zero_rd_en,one_full>	1954	1	-	Covered
1607	Illegal and Ignore Bins:				
1608	ignore_bin full_1	0	-	-	ZERO
1609	ignore_bin full_0	0	-	-	ZERO
1610	Cross wr_rd_empty_cross	100.00%	100	-	Covered
1611	covered/total bins:	8	8	-	
1612	missing/total bins:	0	8	-	
1613	% Hit:	100.00%	100	-	
1614	Auto, Default and User Defined Bins:				
1615	bin <zero_wr_en,zero_rd_en,zero_empty>	1887	1	-	Covered
1616	bin <one_wr_en,zero_rd_en,zero_empty>	4652	1	-	Covered
1617	bin <zero_wr_en,one_rd_en,zero_empty>	675	1	-	Covered
1618	bin <one_wr_en,one_rd_en,zero_empty>	1936	1	-	Covered
1619	bin <zero_wr_en,zero_rd_en,one_empty>	281	1	-	Covered
1620	bin <one_wr_en,zero_rd_en,one_empty>	251	1	-	Covered
1621	bin <zero_wr_en,one_rd_en,one_empty>	221	1	-	Covered
1622	bin <one_wr_en,one_rd_en,one_empty>	97	1	-	Covered
1623	Cross wr_rd_almost_full_cross	100.00%	100	-	Covered
1624	covered/total bins:	8	8	-	
1625	missing/total bins:	0	8	-	
1626	% Hit:	100.00%	100	-	
1627	Auto, Default and User Defined Bins:				
1628	bin <zero_wr_en,zero_rd_en,zero_almostfull>				
1629	bin <one_wr_en,zero_rd_en,zero_almostfull>	1827	1	-	Covered
1630	bin <zero_wr_en,one_rd_en,zero_almostfull>	4539	1	-	Covered
1631	bin <one_wr_en,one_rd_en,zero_almostfull>	710	1	-	Covered
1632	bin <zero_wr_en,zero_rd_en,one_almostfull>	1237	1	-	Covered
1633	bin <one_wr_en,zero_rd_en,one_almostfull>	341	1	-	Covered
1634	bin <zero_wr_en,one_rd_en,one_almostfull>	364	1	-	Covered
1635	bin <one_wr_en,one_rd_en,one_almostfull>	186	1	-	Covered
1636	bin <zero_wr_en,zero_rd_en,zero_almostempty>	796	1	-	Covered
1637	bin <one_wr_en,zero_rd_en,zero_almostempty>	4525	1	-	Covered
1638	bin <zero_wr_en,one_rd_en,zero_almostempty>	809	1	-	Covered
1639	bin <one_wr_en,one_rd_en,zero_almostempty>	1661	1	-	Covered
1640	bin <zero_wr_en,zero_rd_en,one_almostempty>	199	1	-	Covered
1641	bin <one_wr_en,zero_rd_en,one_almostempty>	378	1	-	Covered
1642	bin <zero_wr_en,one_rd_en,one_almostempty>	87	1	-	Covered
1643	bin <one_wr_en,one_rd_en,one_almostempty>	372	1	-	Covered
1644	Cross wr_rd_almost_empty_cross	100.00%	100	-	Covered
1645	covered/total bins:	8	8	-	
1646	missing/total bins:	0	8	-	
1647	% Hit:	100.00%	100	-	
1648	Auto, Default and User Defined Bins:				
1649	bin <zero_wr_en,zero_rd_en,zero_almostempty>				
1650	bin <one_wr_en,zero_rd_en,zero_almostempty>	1969	1	-	Covered
1651	bin <zero_wr_en,one_rd_en,zero_almostempty>	4525	1	-	Covered
1652	bin <one_wr_en,one_rd_en,zero_almostempty>	809	1	-	Covered
1653	bin <zero_wr_en,zero_rd_en,one_almostempty>	1661	1	-	Covered
1654	bin <one_wr_en,zero_rd_en,one_almostempty>	199	1	-	Covered
1655	bin <zero_wr_en,one_rd_en,one_almostempty>	378	1	-	Covered
1656	bin <one_wr_en,one_rd_en,one_almostempty>	87	1	-	Covered
1657	bin <zero_wr_en,zero_rd_en,zero_almostempty>	1969	1	-	Covered
1658	bin <one_wr_en,zero_rd_en,zero_almostempty>	4525	1	-	Covered
1659	bin <zero_wr_en,one_rd_en,zero_almostempty>	809	1	-	Covered
1660	bin <one_wr_en,one_rd_en,zero_almostempty>	1661	1	-	Covered
1661	bin <zero_wr_en,zero_rd_en,one_almostempty>	199	1	-	Covered
1662	bin <one_wr_en,zero_rd_en,one_almostempty>	378	1	-	Covered
1663	bin <zero_wr_en,one_rd_en,one_almostempty>	87	1	-	Covered
1664	bin <one_wr_en,one_rd_en,one_almostempty>	372	1	-	Covered

```

1665 Cross wr_rd overflow_cross 100.00% 100 - Covered
1666 covered/total bins: 6 6 -
1667 missing/total bins: 0 6 -
1668 % Hit: 100.00% 100 -
1669 Auto, Default and User Defined Bins:
1670 bin <zero_wr_en,zero_rd_en,zero_overflow>
1671 2168 1 - Covered
1672 bin <zero_wr_en,one_rd_en,zero_overflow>
1673 896 1 - Covered
1674 bin <one_wr_en,zero_rd_en,zero_overflow>
1675 3461 1 - Covered
1676 bin <one_wr_en,one_rd_en,zero_overflow>
1677 1433 1 - Covered
1678 bin <one_wr_en,zero_rd_en,one_overflow>
1679 1442 1 - Covered
1680 bin <one_wr_en,one_rd_en,one_overflow> 600 1 - Covered
1681 Illegal and Ignore Bins:
1682 ignore_bin ov_1 0 - ZERO
1683 ignore_bin ov_0 0 - ZERO
1684 ✓ Cross wr_rd underflow_cross 100.00% 100 - Covered
1685 covered/total bins: 6 6 -
1686 missing/total bins: 0 6 -
1687 % Hit: 100.00% 100 -
1688 ✓ Auto, Default and User Defined Bins:
1689 ✓ bin <zero_wr_en,zero_rd_en,zero_underflow>
1690 2168 1 - Covered
1691 ✓ bin <zero_wr_en,one_rd_en,zero_underflow>
1692 795 1 - Covered
1693 ✓ bin <one_wr_en,zero_rd_en,zero_underflow>
1694 4903 1 - Covered
1695 ✓ bin <one_wr_en,one_rd_en,zero_underflow>
1696 1819 1 - Covered
1697 ✓ bin <zero_wr_en,one_rd_en,one_underflow>
1698 101 1 - Covered
1699 ✓ bin <one_wr_en,one_rd_en,one_underflow>
1700 214 1 - Covered
1701 ✓ Illegal and Ignore Bins:
1702 ignore_bin under_1 0 - ZERO
1703 ignore_bin under_0 0 - ZERO
1704 Cross wr_rd wrack_cross 100.00% 100 - Covered
1705 covered/total bins: 6 6 -
1706 missing/total bins: 0 6 -
1707 % Hit: 100.00% 100 -
1708 Auto, Default and User Defined Bins:
1709 bin <zero_wr_en,zero_rd_en,zero_wr_ack>
1710 2168 1 - Covered
1711 bin <zero_wr_en,one_rd_en,zero_wr_ack> 896 1 - Covered
1712 bin <one_wr_en,zero_rd_en,zero_wr_ack> 1406 1 - Covered
1713 bin <one_wr_en,one_rd_en,zero_wr_ack> 578 1 - Covered
1714 bin <one_wr_en,zero_rd_en,one_wr_ack> 3497 1 - Covered
1715 bin <one_wr_en,one_rd_en,one_wr_ack> 1455 1 - Covered
1716 Illegal and Ignore Bins:
1717 ignore_bin wrack_1 0 - ZERO
1718 ignore_bin wrack_0 0 - ZERO
1719
1720 TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1
1721
1722 DIRECTIVE COVERAGE:
1723 -----
1724 Name Design Design Lang File(Line) Hits Status
1725 Unit UnitType
1726 -----
1727 /FIFO_top/DUT/cover_Pointer_threshold FIFO Verilog SVA FIFO.sv(192) 7424 Covered
1728 /FIFO_top/DUT/cover_Pointer_Wraparound_read
1729 FIFO Verilog SVA FIFO.sv(191) 161 Covered
1730 /FIFO_top/DUT/cover_Pointer_Wraparound_write
1731 FIFO Verilog SVA FIFO.sv(190) 410 Covered
1732 /FIFO_top/DUT/cover_Almost_empty FIFO Verilog SVA FIFO.sv(189) 984 Covered
1733 /FIFO_top/DUT/cover_Almost_Full FIFO Verilog SVA FIFO.sv(188) 1602 Covered
1734 /FIFO_top/DUT/cover_Full_Flag FIFO Verilog SVA FIFO.sv(187) 2355 Covered
1735 /FIFO_top/DUT/cover_Empty_Flag FIFO Verilog SVA FIFO.sv(186) 815 Covered
1736 /FIFO_top/DUT/cover_Underflow_Detection FIFO Verilog SVA FIFO.sv(185) 238 Covered
1737 /FIFO_top/DUT/cover_Overflow_Detection FIFO Verilog SVA FIFO.sv(184) 1557 Covered
1738 /FIFO_top/DUT/cover_Write_Acknowledge FIFO Verilog SVA FIFO.sv(183) 4701 Covered
1739 /FIFO_top/DUT/cover_Reset_Behavior FIFO Verilog SVA FIFO.sv(182) 483 Covered
1740
1741 TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 11
1742

```

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/coverage_pkg/FIFO_coverage		100.00%	100	100.00...					auto(1)
+ T1FB_FIFO_cvrg_grp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::write_enable_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::read_enable_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::full_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::empty_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::almost_full_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::almost_empty_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::overflow_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::underflow_cp		100.00%	100	100.00...					
+ CVP_FIFO_cvrg_grp::wr_ack_cp		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_full_cross		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_empty_cross		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_almost_full_cross		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_almost_empty_cross		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_overflow_cross		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_underflow_cross		100.00%	100	100.00...					
+ CROSS_FIFO_cvrg_grp::wr_rd_rack_cross		100.00%	100	100.00...					

14. snippets For Assertions:

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
/FIFO_top/DUT/assert__Pointer_threshold	FIFO.sv(180)	0	1
/FIFO_top/DUT/assert__Pointer_Wraparound_read	FIFO.sv(179)	0	1
/FIFO_top/DUT/assert__Pointer_Wraparound_write	FIFO.sv(178)	0	1
/FIFO_top/DUT/assert__Almost_empty	FIFO.sv(177)	0	1
/FIFO_top/DUT/assert__Almost_Full	FIFO.sv(176)	0	1
/FIFO_top/DUT/assert__Full_Flag	FIFO.sv(175)	0	1
/FIFO_top/DUT/assert__Empty_Flag	FIFO.sv(174)	0	1
/FIFO_top/DUT/assert__Underflow_Detection	FIFO.sv(173)	0	1
/FIFO_top/DUT/assert__Overflow_Detection	FIFO.sv(172)	0	1
/FIFO_top/DUT/assert__Write_Acknowledge	FIFO.sv(171)	0	1
/FIFO_top/DUT/assert__Reset_Behavior	FIFO.sv(170)	0	1
/FIFO_top/tb/#ublk#182146786#11/immed__12	FIFO_tb.sv(12)	0	1

Total Coverage By Instance (filtered view): 89.93%

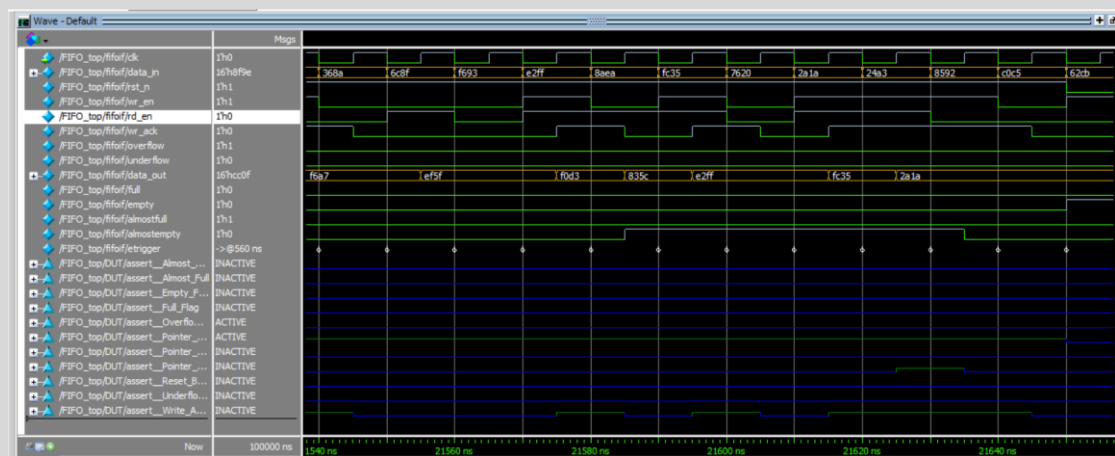
Assertions							
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Acti	
/FIFO_top/DUT/assert__Almost_empty	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Almost_Full	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Empty_Flag	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Full_Flag	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Overflow_Detection	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Pointer_threshold	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Pointer_Wraparound_read	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Pointer_Wraparound_write	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Reset_Behavior	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Underflow_Detection	Concurrent	SVA	on	0	1		
/FIFO_top/DUT/assert__Write_Acknowledge	Concurrent	SVA	on	0	1		
/FIFO_top/tb/#publk#182146786#11/mmed__12	Immediate	SVA	on	0	1		

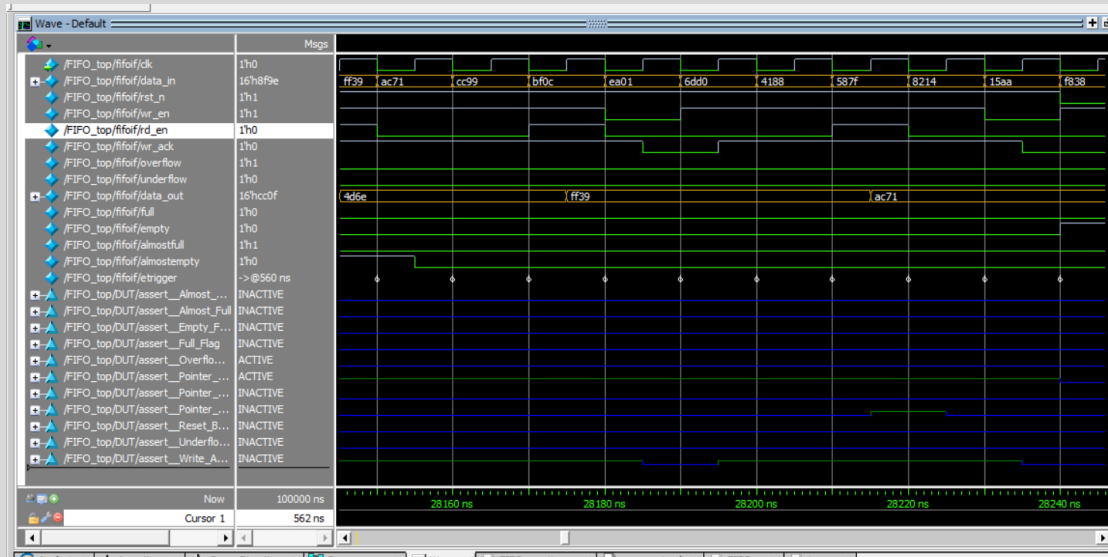
Cover Directives										
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Memory
/FIFO_top/DUT/cover__Pointer_threshold	SVA	✓	Off	7424	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Pointer_Wraparound_read	SVA	✓	Off	161	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Pointer_Wraparound_write	SVA	✓	Off	410	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Almost_empty	SVA	✓	Off	984	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Almost_Full	SVA	✓	Off	1602	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Full_Flag	SVA	✓	Off	2355	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Empty_Flag	SVA	✓	Off	815	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Underflow_Detection	SVA	✓	Off	238	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Overflow_Detection	SVA	✓	Off	1557	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Write_Acknowledge	SVA	✓	Off	4701	1	Unlimited	1	100%	✓	0
/FIFO_top/DUT/cover__Reset_Behavior	SVA	✓	Off	483	1	Unlimited	1	100%	✓	0

15. snippets For waveform:



With assertions:





16. Verification Plan

A	B
Label	Design Requirement Description
FIFO_1	when the reset is asserted ($rst_n=0$), the wr_ptr , rd_ptr , wr_ack , $overflow$ and $underflow$ equal to zero
FIFO_2	there are internal signal named count, this signal doesn't exceed $FIFO_DEPTH$. If the signal wr_en is asserted & count less than $FIFO_DEPTH$, then the $data_in$ will be written in memory wr_ack equal one, wr_ptr will be incremented and count will be incremented
FIFO_3	if the signal rd_en is asserted & count not equal zero, then the $data_out$ equals the data that written in memory and rd_ptr will be incremented and count will be decremented in read operation
FIFO_4	when $count=FIFO_DEPTH$, then the $FULL$ equals one
FIFO_5	when $count=0$, then $empty$ equals one
FIFO_6	when $count=FIFO_DEPTH-1$, then $almost_full$ equals one
FIFO_7	when $count=1$, then $almost_empty$ equals one
FIFO_8	when wr_en is asserted and $full$ equals one, then $overflow$ equals one
FIFO_9	when rd_en is asserted and $empty$ equals one, then $underflow$ equals one
FIFO_10	if wr_en is asserted and rd_en is asserted and the FIFO is full, then the read operation will be done
FIFO_11	if wr_en is asserted and rd_en is asserted and the FIFO is empty, then the write operation will be done

	C
	Stimulus Generation
	directed at the start of sim.then randomized with constraint to make rst is off most of the time
e	wr_en and rd_en are randomized with constraint to make them is one by 70%
	wr_en and rd_en are randomized with constraint to make them is one by 70%
	Directed
	Directed
	Directed
	Directed
	wr_en is randomized with constraint to make it is one by 70%
	wr_en is randomized with constraint to make it is one by 70%
	wr_en and rd_en are randomized with constraint to make them is one by 70%
e	wr_en and rd_en are randomized with constraint to make them is one by 70%

E
Functional coverage2
cover rd_en & wr_en & wr_ack
cover wr_ack, cross cover between wr_en and rd_en and wr_ack. In this cross cover we will ignore the bins that when wr_en=0, rd_en=0, wr_ack=1 and wr_en=0, rd_en=1, wr_ack=1
cover full, cross cover between rd_en and wr_en and full . In this cover we will ignore the bin that wr_en equal 0 and rd_en equal 1 and full equal 1 , and we will ignore the bin that wr_en equal 1 and rd_en equal 1 and full equal 1
cover empty, cross cover between rd_en and wr_en and empty .
cover almost_full, cross cover between rd_en and wr_en and almost_full
cover almost_empty, cross cover between rd_en and wr_en and almost_empty
cover overflow, cross cover between rd_en and wr_en and overflow. In this cross , we will ignore the bins wr_en=0, rd_en=1, overflow=1 and wr_en=0, rd_en=0, overflow=1
cover underflow, cross cover between rd_en and wr_en and underflow. In this cross , we will ignore the bins wr_en=0, rd_en=0, underflow=1 and wr_en=1, rd_en=0, underflow=1

	F	
	Functionality Check	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	
	output is checked against refrence model	

17. The bugs in the Design:

before	after
The underflow was blocking assignment	The underflow will be non blocking assignment and will be assigned in read block
Missing add the state that when $wr_en=1, rd_en=1$ and the FIFO isn't full	Adding the state when $wr_en=1, rd_en=1$ and the FIFO isn't full, then the count will be incremented and the write operation will be done
Missing add the state that when $wr_en=1, rd_en=1$ and the FIFO isn't empty	Adding the state when $wr_en=1, rd_en=1$ and the FIFO isn't empty, then the count will be decremented and the read operation will be done
When the $count=FIFO_DEPTH-2$, then the $almostfull=1$	When the $count=FIFO_DEPTH-1$, then the $almostfull=1$